S.No.	Question	BTL	CO
	UNIT-I		
1	**Set -1** Explain the role of assembler, compiler, loader and linker in the language processing system.	Remember	CO1
2	Explain the role of Lexical Analysis.	Understand	
3	Show the output of each phase in compiler for the following statement. position=initial+rate*60	Apply	
1	**Set 2** Draw a block diagram of phases in compiler and list the main functions of each phase	Remember	
2	Compare and contrast compilers and interpreters	Understand	
3	Write a lexical analyzer program to identify Strings, Sequences, Comments, Reserved words and identifiers	Apply	
1	**Set 3** Explain about the structure of a compiler	Remember	
2	Explain about input buffering in detail.	Understand	
3	Design a transition diagram for identifier and the keyword “else”.	Apply	
	UNIT-II		
1	**Set -1** Explain about ambiguous Grammar with an example	Remember	CO2
2	Construct a recursive descent parser for the following grammar. E->E+T|T T->TF|F F->F*|a|b	Apply	
3	Consider the grammar given below E->E+E|E-E|E*E|E/E|a|b Obtain leftmost and rightmost derivation for the string *a+b*a+b*	Apply	
1	**Set -2** Explain about LL(1) grammars in detail.	Understand	
2	Explain the role of parser in the compiler design.	Understand	
3	Construct predictive parsing table for the following grammar.	Apply	

	E->TE' E'->+TE'|e T->FT' T'->*FT'|e F->(E)|id		
1	**Set -3** Explain about parse tree with an example.	Remember	
2	Explain about the working of LL(1) parser with a neat diagram	Understand	
3	Consider the following grammar E->TE` E`->+TE`|e T->FT` T->*FT`|e F->(E)|id find the FIRST and FOLLOW functions for the above grammar	Apply	
UNIT-II			
1	**Set -1** Explain about Shift Reduce Parsing in detail.	Remember	
2	Explain about LR parsers in detail.	Remember	
3	**Set -2** Compare and contrast LR Parsers and LL Parsers	Understand	
4	Write the procedure for construction of canonical LR parsing table.	Understand	
5	**Set -3** Consider the following grammar E->E+T|T T->TF|F F->F*|a|b Construct the SLR parsing table and parse the input *a*b+a*	Apply	CO3
6	Construct LALR parsing table for the following grammar. S->Aa S->bAc S->dc S->bda A->d Parse the input string bdc.	Apply	
UNIT-III			
1	**Level-1 (Low):** Explain in brief about Synthesized and Inherited Attributes	Remember	CO4

2	What is an Abstract syntax tree? How to construct it using mknode(), mkleaf() functions? Give an example.	Remember	
3	**Level-2 (Medium):** Draw the dependency graph for the following grammar with the input string **int a,b,c** S->T list T-> int T->float T->char T->double List->List,id List->id	Apply	
4	Write the semantic rules for the following grammar. S -> TL T-> int T->float T->char T->double L->L,id L->id	Apply	
5	**Level-3 (High):** Consider the following input statement and represent it in three address code, Quadruples a=b*-c+b*-c	Apply	
6	Consider the following input statement and represent it in triples, indirect triples a=b*-c+b*-c	Apply	
UNIT-IV			
1	**Level-1 (Low):** Define Symbol table. Explain about the data structures for Symbol table.	Remember	
2	Explain about peephole optimization techniques in detail.	Remember	
3	**Level-2 (Medium):** Draw and explain the Runtime memory organization static storage allocation strategy with pros and cons.	Understand	
4	Explain how names can be stored in symbol table with example.	Understand	CO5

5	**Level-3 (High):**
	Construct basic blocks and data flow graph for the following code snippet
	for (i=1 to n)
	{
	j=1;
	while(j<=n)
		Apply	

	{
	A=B*C/D;
	j=j+1;
	}
	}		

6	Consider the following program code for computing dot product of two vectors a and b of length 10 and partition it into basic blocks.
	prod=0;
	i=1;
	do
	{
	prod=prod+a[i]*b[i];
	i=i+1;
	} while(i<=10);
		Apply	

	UNIT-V		
1	**Level-1 (Low):** Explain about the sources and criterions of code optimization as machine dependent and independent types	Remember	
2	Explain common sub expression elimination with an example.	Understand	
3	**Level-2 (Medium):** Explain about i) Instruction Scheduling ii) Elimination of Loop invariant variable with an example	Understand	
4	Explain the following with an example a) Dead code elimination b) Copy propagation, constant folding. c) Strength Reduction	Understand	CO5
5	**Level-3 (High):** Consider the pseudo code for quick sort and perform all the function preserving transformation techniques on flow graph of it.	Apply	

6	Apply loop unrolling optimization technique for the following code snippet
	int i=1;
	while(i<=100)
	{
	a[i]=b[i];
	i++;
	}
		Apply	